

NLP: Key Algorithms & Methods for Exam Questions

Compiled from Course Materials

DS504: Natural Language Processing

September 24, 2025

Question 1: Edit Distance

How do you compute the Levenshtein distance between two strings, X (length n) and Y (length m)?

This is solved using a dynamic programming approach. We define a matrix $D(i, j)$ as the edit distance between the first i characters of X and the first j characters of Y .

Initialization:

$$D(i, 0) = i$$

$$D(0, j) = j$$

Recurrence Relation: For $i = 1 \dots n$ and $j = 1 \dots m$:

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 & \text{(Deletion)} \\ D(i, j-1) + 1 & \text{(Insertion)} \\ D(i-1, j-1) + \text{cost} & \text{(Substitution)} \end{cases}$$

where cost is 0 if $X(i) = Y(j)$, and 1 otherwise (for standard

Question 2: N-gram Language Models

How are probabilities calculated for an n-gram model using Maximum Likelihood Estimation (MLE)?

The probability of a word is conditioned on its preceding context. For n-grams, this context is the previous $n - 1$ words.

General Formula:

$$P(w_i | w_{i-n+1} \dots w_{i-1}) = \frac{C(w_{i-n+1} \dots w_{i-1} w_i)}{C(w_{i-n+1} \dots w_{i-1})}$$

Bigram Example (n=2): The probability of word w_i given the previous word w_{i-1} is:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

where $C(\dots)$ is the count of the n-gram in the training corpus.

How is the probability of a sentence W computed with a bigram model?

Question 3: Smoothing and Evaluation

What is the formula for Laplace (Add-one) Smoothing for bigrams?

To handle zero counts for unseen n-grams, we add one to every count. The denominator is adjusted by adding the vocabulary size V .

$$P_{Add-1}(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

where V is the number of unique words (types) in the vocabulary.

How is the Perplexity (PP) of a language model calculated for a test set W ?

Perplexity is the inverse probability of the test set, normalized by the number of words N . A lower perplexity is better.

$$\begin{aligned} PP(W) &= P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}} \end{aligned}$$

Question 4: Noisy Channel Model

What is the formulation of the Noisy Channel Model for spelling correction?

The goal is to find the intended correct word w given a misspelled observation x . We use Bayes' theorem to find the most probable word.

$$\begin{aligned}\hat{w} &= \arg \max_{w \in V} P(w|x) \\ &= \arg \max_{w \in V} \frac{P(x|w)P(w)}{P(x)} \\ &= \arg \max_{w \in V} \underbrace{P(x|w)}_{\text{Channel Model}} \underbrace{P(w)}_{\text{Language Model}}\end{aligned}$$

- **Channel Model** $P(x|w)$: The probability of observing typo x when the user meant w . Estimated from confusion matrices.
- **Language Model** $P(w)$: The prior probability of the word w . Estimated from a large corpus.

Question 5: HMM and Viterbi for POS Tagging

What are the core probability formulas for an HMM-based POS tagger?

An HMM tagger finds the most likely tag sequence $\hat{T}$ for a word sequence W by maximizing $P(W|T)P(T)$. This simplifies to a product of two types of probabilities:

1. Transition Probability: Probability of a tag given the previous tag.

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

2. Emission (Word Likelihood) Probability: Probability of a word given its tag.

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

What are the recurrence relations for the Viterbi algorithm?

The Viterbi algorithm finds the most probable path through the HMM

Question 6: Parsing Algorithms

What is the general procedure for the CKY parsing algorithm?

CKY is a bottom-up dynamic programming algorithm for parsing with a CFG in Chomsky Normal Form (CNF). It fills an $(n + 1) \times (n + 1)$ table, π .

- 1 **Initialization (Base Case):** For each word w_i , fill cell $\pi[i, i + 1]$ with the non-terminals A for which a rule $A \rightarrow w_i$ exists.
- 2 **Recursive Step:** For increasing span lengths, fill cell $\pi[i, j]$ by finding a split point k (where $i < k < j$) such that there is a rule $A \rightarrow BC$, where $B \in \pi[i, k]$ and $C \in \pi[k, j]$. Add A to $\pi[i, j]$.
- 3 **Termination:** If the start symbol S is in the final cell $\pi[0, n]$, the sentence is accepted by the grammar.

What are the main transitions in the Arc-Eager parsing system?

It's a transition system for deterministic dependency parsing that uses a stack, a buffer, and a set of arcs.

- **SHIFT:** Move the first word from the buffer to the top of the stack. 7/8

Question 7: GloVe Word Embeddings

What is the objective function for the GloVe model?

GloVe (Global Vectors) is a model for learning word embeddings that is based on global co-occurrence statistics. Its objective is a weighted least squares regression model.

Objective Function:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(X_{ij}) \right)^2$$

Components:

- $w_i, \tilde{w}_j$: Word vector and context word vector.
- $b_i, \tilde{b}_j$: Bias terms for each vector.
- X_{ij} : The count of how many times word j appears in the context of word i .
- $f(X_{ij})$: A weighting function that down-weights very frequent co-occurrences to avoid them dominating the training process.